

DepthWizard — Product Requirements Document

SIH26175 | ISRO — Single-View Height Estimation & 3D Flythrough

1. Vision

DepthWizard turns a single optical satellite/aerial image into an interactive 3D terrain surface — reconstructing a Digital Surface Model (DSM) that captures canopy, structure, and terrain relief as visible from above, letting users fly through it, and (where calibration data exists) measuring real elevation, distance, and slope — without requiring stereo pairs, LiDAR, or multi-view capture. It gives analysts, planners, and field teams a fast, offline-capable way to "see in 3D" from imagery alone.

2. Problem Statement

Multi-view/LiDAR-based elevation modeling is accurate but expensive, slow, and often infeasible in the field or in data-scarce regions. There is no lightweight tool that takes one optical image and produces a usable, explorable 3D terrain reconstruction — with honest, clearly-labeled accuracy — in seconds, fully offline.

3. Product Goals

1. Reconstruct a plausible Digital Surface Model (DSM) from a single 2D image using monocular depth inference — capturing visible surface relief (canopy, rooftops, structures) rather than claiming a bare-earth DEM.
 2. Where ground-truth calibration data (GCPs/reference elevation) exists for a region, convert relative depth into metric DSM elevation (meters AMSL) and expose validated accuracy telemetry.
 3. Where no calibration data exists, still deliver an instant, clearly-labeled *relative* 3D DSM reconstruction — never fake precision.
 4. Provide an explorable, cinematic 3D viewer: drone flythrough, point-to-point elevation measurement, water-leveled terrain.
 5. Run fully offline on modest hardware for demo reliability.
-

4. Target Audience

- Hackathon evaluators / ISRO problem-statement judges (primary, for SIH).
 - Terrain/reconnaissance analysts who need quick 3D context from a single image without survey equipment.
 - Disaster-response and field-planning teams operating in low-connectivity environments.
-

5. Success Metrics

Metric	Target
Cold-to-rendered-mesh latency (preset region)	< 3 s end-to-end
Model inference latency (warm, CPU)	~261 ms (Depth-Anything-V2-Small, verified)

Calibrated mode accuracy (preset regions)	Reported honestly via live r / MAE / RMSE HUD
Relative mode availability	100% of arbitrary uploads produce a mesh, no failures
Offline functionality	Full demo runnable with zero network calls
Judge-facing "wow"	Working drone flythrough + elevation ruler on at least 3 curated regions

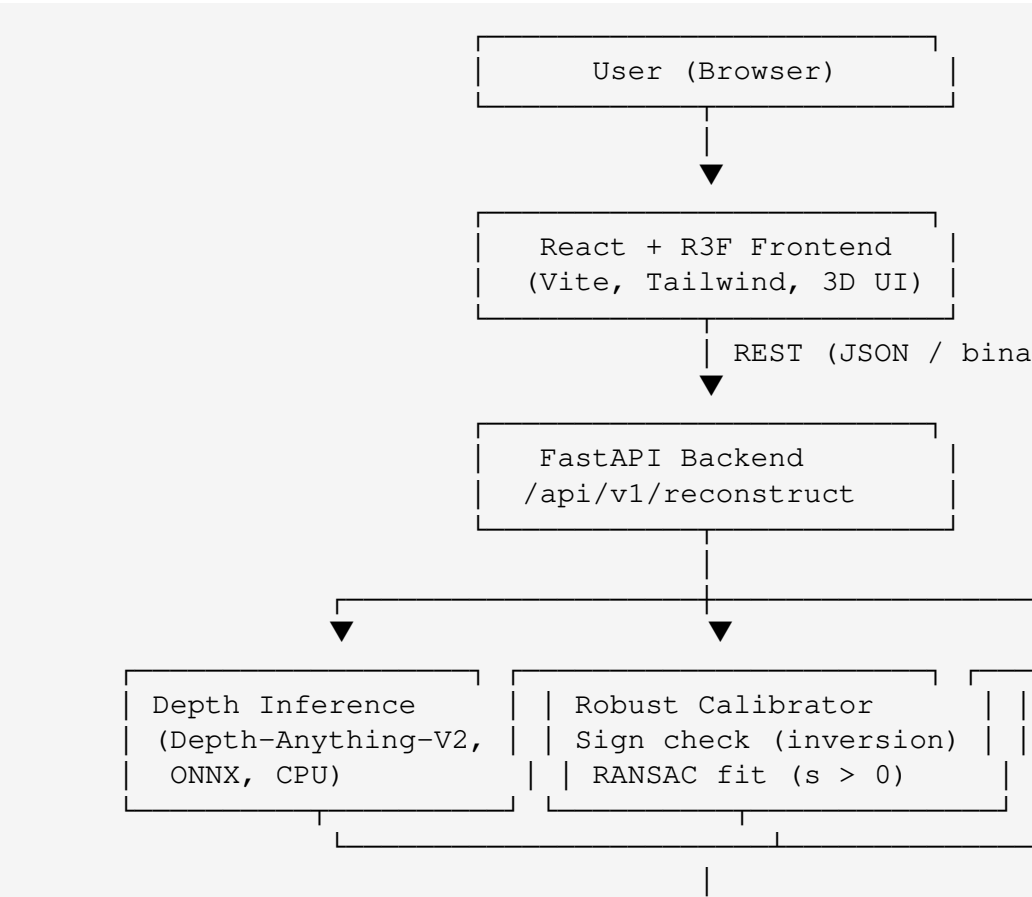
6. Technology Stack

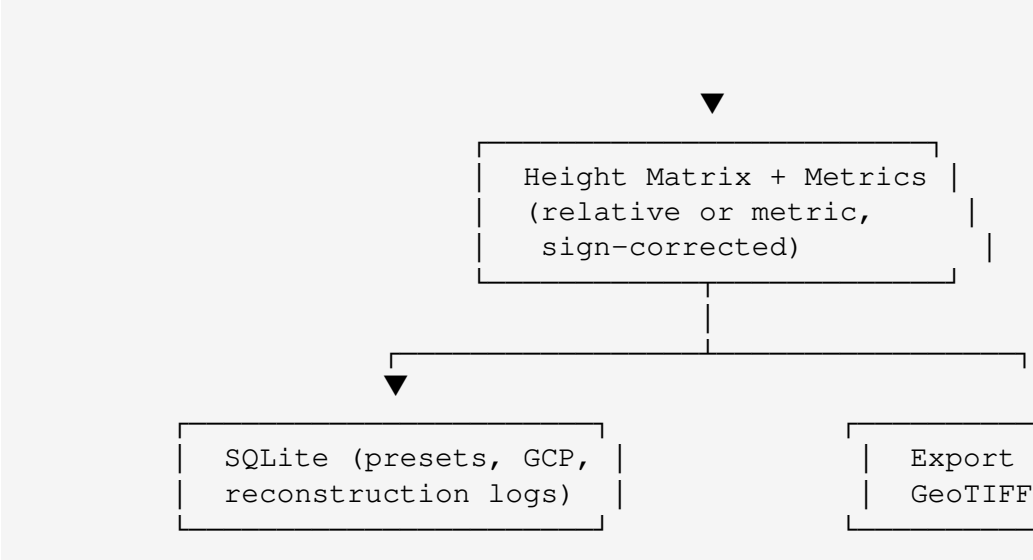
Component	Technology	Responsibility
Backend framework	FastAPI (Python)	API routing, request validation, orchestration
Depth inference	Depth-Anything-V2-Small (ONNX Runtime, CPU)	Zero-shot monocular relative depth prediction
Edge snapping	<code>cv2.ximgproc.guidedFilter</code> — requires <code>opencv-contrib-python-headless</code> (Joint Bilateral alternative)	Refines depth boundaries against optical luminance — snaps rounded/pillowy depth edges to true structural edges

Calibration	scikit-learn <code>RANSACRegressor</code> (or SciPy <code>soft_l1</code> loss)	Outlier-resistant affine fit: $Z = s \cdot \tilde{d} + t$, sign- constrained ($s > 0$)
Water segmentation	opencv-contrib-python- headless (VWI index + Otsu thresholding + luminance gate)	Detect and flatten water bodies while excluding dark shadow regions
Height transport	16-bit grayscale PNG / Float32Array (Phase 2)	Binary heightfield transport, GPU- side shader displacement
Geo export	rasterio	GeoTIFF export of metric height maps
Mesh export	trimesh / pygltflib	GLB export of 3D mesh
Data storage	SQLite	Preset metadata, GCP anchors, reconstruction/ validation logs
Frontend framework	React + Vite	UI shell, state, routing
3D rendering	React Three Fiber (Three.js)	PlaneGeometry mesh, texture draping, camera/spline flythrough
Styling	Tailwind CSS	Layout and visual design

Charts	Recharts	Elevation cross-section profile, validation stats
Dev hardware target	Intel Core Ultra 5 125H, 16GB RAM	CPU-only inference, no dedicated GPU assumed

7. System Architecture





8. Database Design

presets

Field	Type	Notes
id	TEXT (PK)	e.g. leh, hyderabad, assam
name	TEXT	Display name
bbox	TEXT (JSON)	Bounding box coordinates
tile_path	TEXT	Path to bundled optical tile
dem_path	TEXT	Path to bundled reference DEM (nullable)

ground_resolution_m_per_px	REAL NOT NULL	Ground sampling distance — required for any real-world distance/slope math
created_at	DATETIME	

gcp_anchors

Field	Type	Notes
id	INTEGER (PK)	
preset_id	TEXT (FK → presets.id)	
pixel_x	INTEGER	
pixel_y	INTEGER	
elevation_m	REAL	Ground-truth elevation (AMSL)

reconstructions

Field	Type	Notes
id	TEXT (PK, UUID)	
preset_id	TEXT (FK → presets.id, nullable)	Null if arbitrary upload
mode	TEXT	tactical relative
source_image_path	TEXT	
height_matrix_path	TEXT	Serialized array (numpy/json)

affine_s	REAL	Nullable — only for tactical mode; enforced > 0 (post inversion-correction)
affine_t	REAL	Nullable — only for tactical mode
depth_inverted	BOOLEAN	Whether the raw depth map was sign-flipped before fitting
ground_resolution_m_per_px	REAL	Copied from preset or derived from upload metadata; required for ruler math
metric_r	REAL	Correlation vs GCP/DEM ground truth (post sign-correction)
metric_mae	REAL	
metric_rmse	REAL	
created_at	DATETIME	

elevation_profiles

Field	Type	Notes
id	INTEGER (PK)	
reconstruction_id	TEXT (FK → reconstructions.id)	
point_a_x	INTEGER	
point_a_y	INTEGER	
point_b_x	INTEGER	
point_b_y	INTEGER	

distance_m	REAL	
elevation_delta_m	REAL	
slope_percent	REAL	

Relationships: presets 1—N gcp_anchors; presets 1—N reconstructions; reconstructions 1—N elevation_profiles.

9. Phased Feature Breakdown

Phase 1 — MVP (Backend Pipeline)

9.1 Core Inference Route

- FastAPI project scaffold with pinned requirements.txt.
- /api/v1/reconstruct accepts either preset_id, a bbox, or an arbitrary uploaded image.
- Runs Depth-Anything-V2-Small on CPU, returns normalized relative depth (2%–98% percentile contrast stretch) representing a raw DSM surface (not bare-earth).

9.1a Edge-Snapping (Guided Filter)

- Immediately after depth normalization, run an OpenCV Guided Filter (cv2.ximgproc.guidedFilter, or Joint Bilateral Filter as fallback) using the optical tile luminance as the guidance image.
- Purpose: raw monocular depth tends to produce rounded, "pillowy" boundaries at structural edges (building walls, cliff faces); the guided filter snaps these transitions to the true visual edges in the optical image, producing crisper vertical drops consistent with the DSM's actual surface discontinuities.
- Runs before calibration/inversion-check, so both tactical and relative modes benefit from sharper geometry.

9.2 Dual-Mode Route Contract

- **Tactical Mode** (preset_id or valid bbox with bundled DEM/GCPs present):
 - **Depth inversion check (mandatory, run first):** compute Pearson $r(\hat{d}, \text{GCP}_z)$. If $r < 0$, invert depth ($\hat{d} \leftarrow 1 - \hat{d}$) before fitting — monocular models predict depth-from-camera, not elevation-above-ground, and nadir imagery can invert the gradient (ridges vs. valleys) depending on illumination/shadow. Skipping this check risks rendering inverted terrain.
 - **Robust affine fit:** replace ordinary least-squares with `sklearn.linear_model.RANSACRegressor` (or SciPy `soft_l1` loss), constrained to $s > 0$ post-inversion-correction. A handful of noisy GCPs must not wreck the whole calibration.
 - Returns metric height in meters AMSL plus validation telemetry (r , MAE, RMSE), all computed post sign-correction.
- **Relative Reconnaissance Mode** (arbitrary/unreferenced upload): returns normalized $[0, 100]$ relative height mesh, metric telemetry HUD disabled — never fabricate precision.

9.3 Water Body Leveling

- Visible Water Index (VWI) computed on the optical tile. For arbitrary RGB inputs with no NIR band available, use the 3-band proxy:

$$\text{VWI} = (\text{Green} - \text{Red}) / (\text{Green} + \text{Red} + 1\text{e-}5)$$

- Otsu thresholding to segment water pixels.
- **Luminance gate** ($L > \tau$) applied on top of the VWI+Otsu mask — prevents dark terrain (mountain cloud shadows, building shadows, deep ravines) from being misclassified as water and incorrectly flattened.
- Water pixels clamped to a single baseline elevation to remove artificial surface waviness.

9.4 Offline Preset Bundling

- Leh, Hyderabad, Assam tiles + DEMs + GCP anchors pre-

packaged as local static assets so the full demo runs with zero network calls.

Phase 2 — Enhancements (Frontend + Tactical Tools)

9.5 3D Viewer

- Vite + React + Tailwind frontend.
- React Three Fiber viewport, 256×256 `PlaneGeometry`, vertex displacement from the returned height matrix, optical image draped as texture.

9.6 Cinematic Flythrough

- Camera follows a **progressive, directional** Catmull-Rom spline path that sweeps forward through terrain corridors — simulating a forward reconnaissance drone pass over the reconstructed DSM.
- Explicitly **not** a circular turntable orbit around the mesh center — the path must have a clear entry and exit direction consistent with a reconnaissance sweep, not a static rotate-in-place camera.

9.7 Tactical Analysis Tools

- Point A / Point B selection is **exclusively** via direct 3D raycasting/click interaction on the mesh surface — extracting UV coordinates mapped directly to pixel indices on the underlying height matrix.
- **Manual coordinate input boxes are explicitly prohibited** for point selection — the tool must feel like pointing at the terrain, not typing pixel coordinates.
- **Mandated UV-to-matrix transform** (prevents vertical coordinate inversion bugs — texture V-axis and array row-axis run in opposite directions):

```
col = int(uv.x * (width - 1))  
row = int((1.0 - uv.y) * (height - 1))
```

- Recharts line-profile chart: distance, altitude delta, slope % — computed using `ground_resolution_m_per_px`, without which distance/slope numbers are not real measurements.

- Validation HUD: live r / MAE / RMSE for calibrated (tactical) reconstructions.

9.8 Export

- GeoTIFF export (`rasterio`) for metric height maps.
- GLB mesh export (`pygltflib/trimesh`) for the 3D model.

9.9 Binary Payload Optimization (Phase 2, not MVP-blocking)

- Replace JSON-serialized height matrix with a 16-bit grayscale PNG or raw `Float32Array` response — cuts payload size by >80% vs. base64 JSON and removes serialization overhead.
- Frontend decodes directly into a `DataTexture`, with vertex displacement moved into a GLSL shader (GPU-side) instead of CPU-side JS geometry generation:

```
float height = texture2D(u_heightmap, vUv).r;  
vec3 displacedPosition = position + normal * (height * u_
```

- Ship the MVP with JSON transport first; swap to binary once the pipeline is validated end-to-end — don't let this gate Phase 1 delivery.

10. AI Pipeline

Optical Image



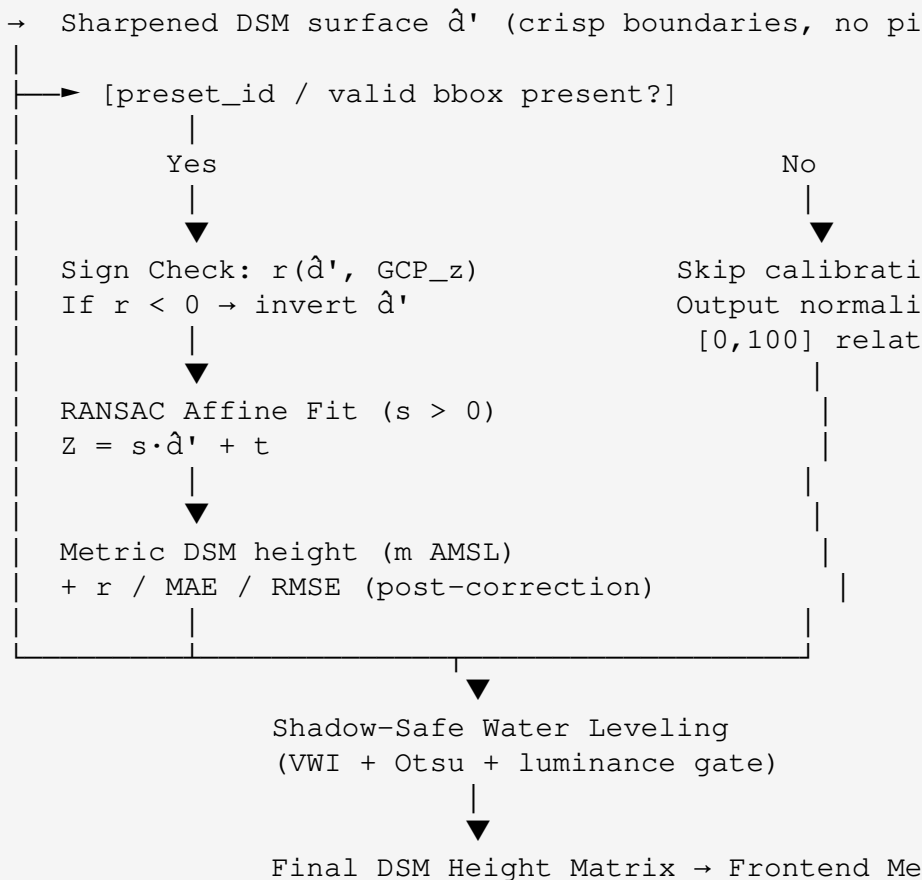
Depth-Anything-V2-Small (zero-shot monocular inference)



Percentile Contrast Stretch (2%-98%) → Normalized relat



Guided Filter Edge-Snapping (optical luminance as guide)



Sample response — Tactical Mode

```

{
  "mode": "tactical",
  "preset_id": "leh",
  "height_matrix": "base64-or-path-reference",
  "units": "meters_amsl",
  "ground_resolution_m_per_px": 10.0,
  "calibration": { "s": 142.7, "t": -18.3, "depth_inverted": true },
  "metrics": { "r": 0.41, "mae": 24.6, "rmse": 31.2 },
  "water_mask_applied": true
}

```

Sample response — Relative Reconnaissance Mode

```
{
  "mode": "relative",
  "preset_id": null,
  "height_matrix": "base64-or-path-reference",
  "units": "relative_0_100",
  "ground_resolution_m_per_px": null,
  "calibration": null,
  "metrics": null,
  "water_mask_applied": true
}
```

Rules

- `calibration` and `metrics` are `null` in relative mode — the frontend MUST hide the telemetry HUD when either is `null`.
- `calibration.depth_inverted` must be checked before the sign-correction step is trusted; it exists so the frontend/QA can verify the inversion check actually ran, not just assume it.
- `ground_resolution_m_per_px` is `null` in relative mode — the elevation ruler tool MUST disable real-unit distance/slope output (fall back to pixel-relative units, or hide the ruler) whenever this is `null`.
- `units` always explicitly states whether values are metric or relative — no ambiguous output.
- `water_mask_applied` always present so the frontend can toggle a "water leveled" indicator.

11. UI Principles

- **Never fake precision.** Relative mode visually and textually distinguishes itself from tactical mode (badge/label, disabled metric HUD).
- **The mesh is the hero.** Minimal chrome around the 3D viewport; controls (flythrough toggle, ruler tool, export) as compact floating

panels.

- **Instant feedback.** Loading states for inference (~261 ms target) should never feel like a stall — use a lightweight progress indicator, not a spinner-only screen.
 - **Judges should understand accuracy at a glance.** Validation HUD (r/MAE/RMSE) always visible in tactical mode, never buried in a settings panel.
-

12. Folder Structure

```
depthwizard/
├── backend/
│   ├── app/
│   │   ├── main.py
│   │   ├── routers/
│   │   │   └── reconstruct.py
│   │   ├── services/
│   │   │   ├── inference.py           # Depth-Anything-V2
│   │   │   ├── calibration.py        # Affine GCP fit
│   │   │   ├── water_leveling.py     # VWI + Otsu
│   │   │   └── export.py             # GeoTIFF / GLB expo
│   │   ├── models/
│   │   │   └── db.py                 # SQLite schema/ORM
│   │   └── data/
│   │       └── presets/
│   │           ├── leh/
│   │           ├── hyderabad/
│   │           └── assam/
│   └── requirements.txt
├── frontend/
│   ├── src/
│   │   ├── components/
│   │   │   ├── Viewer3D.jsx
│   │   │   ├── FlythroughControls.jsx
│   │   │   ├── ElevationRuler.jsx
│   │   └── ValidationHUD.jsx
```

```
|
|
|   |   |   | pages/
|   |   |   |   | App.jsx
|   |   |   |   | lib/
|   |   |   |   |   | api.js
|   |   |   | index.html
|   |   |   | package.json
|   |   |   | README.md
```

13. Environment Variables

Variable	Purpose	Example
MODEL_PATH	Path to Depth-Anything-V2-Small weights	<code>./data/models/da_v2_small.onnx</code>
PRESET_DATA_DIR	Root path for bundled preset tiles/DEMs/GCPs	<code>./data/presets</code>
DB_PATH	SQLite database file path	<code>./data/depthwizard.db</code>
MAX_UPLOAD_MB	Max accepted image upload size	20
INFERENCE_DEVICE	Force CPU/GPU device selection	cpu
VITE_API_BASE_URL	Backend base URL for frontend	<code>http://localhost:8000</code>

14. Non-Functional Requirements

- **Offline-first:** All preset-mode functionality must work with zero network access.
 - **Performance:** Warm inference ≤ 300 ms on target CPU; total request round-trip ≤ 3 s.
 - **Honesty by design:** No UI path may display metric units/telemetry when calibration data is absent.
 - **Portability:** Backend and frontend runnable via standard `pip/npm` install with no proprietary dependencies.
 - **Resilience:** Arbitrary/unsupported image uploads must degrade gracefully to relative mode, never error out.
-

15. Future Scope (Out of Scope for Hackathon)

- Live/online DEM fetching for arbitrary bounding boxes beyond bundled presets.
 - User accounts, multi-user session history.
 - Model fine-tuning on India-specific terrain datasets.
 - Mobile/field-device native app.
 - Automated GCP discovery from public survey data (removing manual anchor curation).
-

16. Final Product Flow

User selects a preset OR uploads an image



Frontend calls `/api/v1/reconstruct`



Backend runs depth inference (Depth-Anything-V2-Small)



